WorkCore Workflow Model Authoring Guide

Version: 1.0
Date: 2026-03-04
Audience: LLM agents that design workflow graphs and hand off implementation-ready payloads.

1) Purpose

This document is a single model-facing contract for creating WorkCore workflows that are:

- valid in Builder,
- publishable by runtime rules,
- executable with predictable behavior,
- easy to implement via API.

Primary source artifacts used to compile this guide:

- `docs/architecture/workflow-authoring-agents.md`
- `docs/architecture/node-semantics.md`
- `docs/architecture/runtime.md`
- `docs/architecture/state-and-expressions.md`
- `docs/architecture/executors.md`
- `docs/architecture/streaming.md`
- `docs/architecture/overview.md`
- `docs/api/openapi.yaml`
- `docs/api/schemas/workflow-draft.schema.json`
- `docs/api/schemas/workflow-export-v1.schema.json`
- `apps/builder/src/builder/types.ts`
- `apps/builder/src/builder/graph.ts`

2) Hard Constraints (Do Not Violate)

Graph invariants:

- Exactly one `start` node.
- At least one `end` node.
- Node IDs must be unique.
- Every edge must reference existing node IDs.
- There must be at least one path from `start` to an `end` node.
- All nodes must be reachable from `start`.

Node-specific validation:

- `if_else` must include non-empty `branches[]` with `{ condition, target }`.

- `while` must define `condition`, `max_iterations`, `body_target`, `exit_target`, `loop_back`.
- `set_state` must provide either:
  - legacy mode: `target` + `expression`, or
  - batch mode: non-empty `assignments[]` with `{ target, expression }`.
- `integration_http` must define `url`.

Publish/runtime guardrails:

- Do not invent node types beyond supported enum.
- Do not invent config fields outside documented payloads.
- Use idempotency keys for mutating run/interrupt/cancel/rerun calls.

3) Supported Node Types

Allowed node types:

- `start`
- `agent`
- `mcp`
- `integration_http`
- `if_else`
- `while`
- `set_state`
- `interaction`
- `approval`
- `output`
- `end`

4) Node Semantics and Minimum Configuration

`start`

- Purpose: validate inputs, apply defaults, initialize run state.
- Typical config:
  - `defaults` (optional object).
- Workflow-level `variables_schema` should describe expected input shape.

`agent`

- Purpose: execute LLM task through OpenAI Agents SDK.
- Minimum practical config:
  - `instructions` (non-empty recommended).
- Common optional fields:
  - `model`, `allowed_tools`, `output_format`, `output_schema`, `state_target`,
  - `merge_output_to_state`, `max_retries`, `timeout_s`, `emit_partial`.

- Output behavior:
  - always persisted into `node_runs.output` and `node_outputs[node_id]`.
  - if `state_target` is set: full output written there.
  - if structured JSON output and no `state_target`: top-level keys auto-merge into state unless `merge_output_to_state=false`.

`mcp`

- Purpose: call tool through MCP bridge (`POST /internal/mcp/call`).
- Minimum practical config:
  - `server`, `tool`.
- Optional fields:
  - `arguments`, `timeout_s`, `allowed_tools`.

`integration_http`

- Purpose: direct external HTTP call from runtime.
- Required:
  - `url`.
- Optional fields:
  - `method` (`GET|POST|PUT|PATCH|DELETE`)
  - `headers`, `auth`, `timeout_s`, `retry_attempts`, `retry_backoff_s`
  - `request_body_expression`
  - `response_state_target`, `response_body_state_target`
  - `fail_on_status`, `allowed_statuses`.
- Runtime egress is deny-by-default and controlled by env allowlists.

`if_else`

- Purpose: conditional routing.
- Required:
  - non-empty `branches[]` where each item has `condition` and `target`.
- Optional:
  - `else_target`.

`while`

- Purpose: bounded loop with explicit cycle path.
- Required:
  - `condition`, `max_iterations`, `body_target`, `exit_target`, `loop_back`.

`set_state`

- Purpose: compute/update state variables.
- Config modes:

  - legacy single assignment: `target` + `expression`
  - batch assignment: `assignments[]`.
- Batch assignments execute in order and later expressions can reference earlier updates.

`interaction`

- Purpose: collect user input via interrupt.
- Minimum practical config:
  - `prompt` (non-empty recommended).
- Optional:
  - `allow_file_upload`, `input_schema`, `state_target`.

`approval`

- Purpose: approval interrupt (approve/reject).
- Minimum practical config:
  - `prompt` (non-empty recommended).
- Optional:
  - `allow_file_upload`, `state_target`.

`output`

- Purpose: produce final output payload.
- Config style:
  - expression-based via `expression`, or
  - static via `value`.

`end`

- Purpose: mark run completion.
- Required config: none.

5) State and Expression Contract

Expression context keys:

- `inputs`
- `state`
- `node_outputs`

CEL examples:

- `state.intent == "refund"`
- `inputs.customer_text != ""`
- `node_outputs["score_node"].score > 0.8`

Type handling:

- `start` validates `inputs` against `variables_schema`.
- `set_state` validates assigned values against declared variable schema.

- Missing output references in templating are treated as invalid arguments.

6) Runtime Lifecycle Contract

Run statuses:

- `RUNNING`
- `WAITING_FOR_INPUT`
- `COMPLETED`
- `FAILED`
- `CANCELLED`

Node run statuses:

- `TO_DO`
- `IN_PROGRESS`
- `RESOLVED`
- `ERROR`
- `CANCELLED`

Semantics:

- Scheduler runs node when inbound dependencies are resolved.
- Non-selected `if_else` branches are skipped.
- `while` condition is re-evaluated each cycle.
- Node retries use attempt increments; on exhaustion run fails.
- `interaction` / `approval` create interrupts and transition run to `WAITING_FOR_INPUT`.
- Resume interrupt continues scheduling.
- Run pins published workflow version (`resolved_version`) at start; resume uses pinned version.

7) Draft JSON Contract for Builder/API

Use `WorkflowDraft` shape only:

```
{
  "nodes": [
    { "id": "start", "type": "start", "config": { "defaults": {} } },
    { "id": "end", "type": "end", "config": {} }
  ],
  "edges": [
    { "source": "start", "target": "end" }
  ],
  "variables_schema": {}
}
```

Notes:

- `config.ui` is managed by Builder import/export and should not be fabricated unless round-tripping Builder JSON.

- Machine validation schemas:
  - `docs/api/schemas/workflow-draft.schema.json`
  - `docs/api/schemas/workflow-export-v1.schema.json`

8) API Sequence to Implement a Workflow

Required headers for workflow APIs:

- `Authorization: Bearer <token>`
- `X-Tenant-Id: <tenant_id>`
- `X-Project-Id: <project_id>`

Recommended headers:

- `X-Correlation-Id`
- `X-Trace-Id`
- `Idempotency-Key` for mutating calls

Typical sequence:

1. `POST /workflows` with `name`, optional `description`, optional `draft`.
2. `PUT /workflows/{workflow_id}/draft` with full `WorkflowDraft`.
3. `POST /workflows/{workflow_id}/publish`.
4. `POST /workflows/{workflow_id}/runs` with `inputs` and optional `mode` (`live|test|sync|async`).
5. Observe run:
   - `GET /runs/{run_id}` (polling)
   - `GET /runs/{run_id}/stream` (SSE)
6. Handle interrupts:
   - `POST /runs/{run_id}/interrupts/{interrupt_id}/resume`
   - `POST /runs/{run_id}/interrupts/{interrupt_id}/cancel`
7. Operations:
   - `POST /runs/{run_id}/cancel`
   - `POST /runs/{run_id}/rerun-node`

Useful validation endpoint for model-generated drafts:

- `POST /agent-integration-test/validate-draft`

9) Event Streaming Contract (SSE)

Endpoint:

- `GET /runs/{run_id}/stream`

Event types include:

- `snapshot`
- `run_started`
- `node_started`

- `node_completed`
- `node_failed`
- `node_retry`
- `run_waiting_for_input`
- `run_completed`
- `run_failed`
- `message_generated`
- `stream_end`

Reconnect semantics:

- Send `Last-Event-ID` to replay from the next event.
- Without `Last-Event-ID`, stream sends one `snapshot` first.

10) Document Payload and Projection Rules

For document workflows:

- Prefer `inputs.documents[].pages[].artifact_ref` instead of inline heavy payload.
- Use run projections to control payload size:
  - `state_exclude_paths`
  - `output_include_paths`
- Read full content explicitly when needed via:
  - `GET /artifacts/{artifact_ref}`

11) Model Output Package (What the Model Must Produce)

For each requested workflow, the model should output all items below.

1. Objective card:

- business goal
- success criteria
- required inputs
- final output contract
- failure policy

2. Graph plan:

- node list with IDs and types
- edge list
- branch/loop target mapping

3. Draft payload:

- valid `WorkflowDraft` JSON only
- no extra node types
- no undocumented required fields

4. Config completeness check:

- verify all required node config fields

- verify graph invariants

5. API execution plan:

- exact endpoint sequence for create/draft/publish/run
- required headers
- idempotency plan

6. Smoke-test plan:

- sample `inputs`
- expected lifecycle transitions
- expected completion or interrupt behavior

12) Action Items Template (for Reliable Delivery)

Use this checklist before handoff:

1. Define workflow objective and measurable success condition.
2. Confirm required inputs and `variables_schema`.
3. Build minimal path `start -> ... -> end`.
4. Add branching/looping only when required by objective.
5. Fill node configs with supported fields only.
6. Validate graph invariants and node-level required fields.
7. Publish version.
8. Run at least one `mode=test` smoke run.
9. Capture run events/logs and verify expected transitions.
10. Record open TODOs for missing business decisions instead of inventing values.

13) Do / Don't

Do:

- Keep workflows minimal and explicit.
- Use deterministic routing targets (`if_else`, `while`) with concrete node IDs.
- Use `set_state` to normalize data for downstream conditions and outputs.
- Use idempotency keys on mutating API calls.

Don't:

- Do not invent custom node types.
- Do not publish with validation errors.
- Do not rely on implicit branching or hidden side effects.
- Do not embed secrets inline in workflow configs when env refs are supported.

14) Canonical Source Map

Architecture and semantics:

- `docs/architecture/overview.md`
- `docs/architecture/runtime.md`

- `docs/architecture/node-semantics.md`
- `docs/architecture/state-and-expressions.md`
- `docs/architecture/executors.md`
- `docs/architecture/streaming.md`
- `docs/architecture/workflow-authoring-agents.md`

API and schemas:

- `docs/api/openapi.yaml`
- `docs/api/schemas/workflow-draft.schema.json`
- `docs/api/schemas/workflow-export-v1.schema.json`

Builder implementation references:

- `apps/builder/src/builder/types.ts`
- `apps/builder/src/builder/graph.ts`